

GillNet AI — Backend Security & Code Audit

Comprehensive Architectural, Security, and Code Quality Review

TARGET CODEBASE	AUDIT MODE	SCOPE	DATE / STATUS
GillNet-AI-main (Spring Boot & ML Service)	Read-Only Static Analysis & Inspection	Auth, APIs, Java Backend, Python ML	September 2026 / Completed

3

CRITICAL SECURITY FLAWS

2

BUILD / TEST FAILURES

6

MISSING CORE ENDPOINTS

5

ML SERVICE BUGS

1. Executive Summary

GillNet AI is intended to serve as a personal cybersecurity web suite providing phishing URL detection, scam message analysis, password strength evaluation, and an AI cybersecurity assistant. A thorough code audit of the repository reveals that the backend is currently in an early, incomplete state with **severe security vulnerabilities**, **broken build tests**, and **zero implementation of core cybersecurity endpoints**.

Most notably, user passwords are saved in **unhashed plaintext**, exposed via public endpoints, the Spring Boot test suite fails out-of-the-box due to mismatched package names, and the React frontend operates on disconnected client-side heuristics without any backend API communication.

2. Audit Findings Matrix

SEVERITY	ISSUE TITLE	LOCATION	IMPACT
CRITICAL	Plaintext Password Storage	UserService.java:20-31	CWE-256 / CWE-312: Unhashed DB credentials
CRITICAL	Public Credential Leak Endpoint	UserController.java:30-36	Unauthenticated password harvesting via GET API
CRITICAL	Password Leaked in Registration Response	UserController.java:20-28	Plaintext password returned to frontend client
CRITICAL	Test Suite Fails Out-of-the-Box	CybershieldAiApplicationTests.java	Package mismatch breaks ./mvnw test
HIGH	Missing Configuration Files	src/main/resources/ (Missing)	No DB connection string or port settings
HIGH	Missing Core Cybersecurity APIs	com.gillnet.controller.*	No URL, message, password, or chat APIs
HIGH	Path-Dependent Pickle Loading	ml-service/app.py:14-19	Crashes if run outside ml-service/ dir
HIGH	Valid URLs Rejected as Invalid	ml-service/app.py:281-289	Rejects URLs missing http:// / https://
MEDIUM	Contradictory ML Response & Scoring	ml-service/app.py:348-371	Static risk score ignores model confidence
MEDIUM	Inaccurate Feature Extraction Logic	ml-service/app.py:94-143	Flawed @ , subdomain, & redirect checks

SEVERITY	ISSUE TITLE	LOCATION	IMPACT
MEDIUM	Missing Dataset phishing.csv	ml-service/train_url_model.py:11	Model retraining script fails with FileNotFoundError

3. Detailed Findings: Security Vulnerabilities (Critical)

CRITICAL Plaintext Password Storage & Absence of Hashing

CWE-256 / CWE-312

com.gillnet.service.UserService.java (Lines 20-31) | com.gillnet.model.User.java

The `UserService.registerUser()` method directly persists the incoming `User` object without hashing the password. There is no `PasswordEncoder` configured anywhere in the Spring context.

```
// UserService.java:20-31
public User registerUser(User user) {
    Optional<User> existingUser = userRepository.findByEmail(user.getEmail());
    if (existingUser.isPresent()) {
        throw new RuntimeException("Email already registered");
    }
    user.setCreatedAt(LocalDateTime.now());
    return userRepository.save(user); // Raw password saved directly to MongoDB!
}
```

Remediation: Inject a `BCryptPasswordEncoder` bean into `UserService` and hash passwords before persisting:
`user.setPassword(passwordEncoder.encode(user.getPassword()));`

CRITICAL Public Credential & Password Exposure Endpoint

CWE-200 / CWE-359

com.gillnet.controller.UserController.java (Lines 30-36)

The endpoint `GET /api/users/email/{email}` is publicly accessible without any authentication. It directly serializes the internal `User` MongoDB document into the HTTP response, transmitting the user's password in plain text over the network.

```
// UserController.java:30-36
@GetMapping("/email/{email}")
public ResponseEntity<?> getUserByEmail(@PathVariable String email) {
    return userService.findByEmail(email)
        .map(ResponseEntity::ok) // DANGEROUS: Returns entire User object with password!
        .orElse(ResponseEntity.notFound().build());
}
```

Remediation: Immediately delete or restrict this endpoint. Never return raw entity models from controllers; use a sanitized `UserResponseDTO` containing only safe public attributes (id, name, email, createdAt).

CRITICAL**Registration Response Leaks Password**

CWE-200

com.gillnet.controller.UserController.java (Lines 20-28)

The `POST /api/users/register` endpoint returns `ResponseEntity.ok(savedUser)`, echoing back the user's plaintext password in the JSON body.

```
// UserController.java:21-25
@PostMapping("/register")
public ResponseEntity<> registerUser(@RequestBody User user) {
    User savedUser = userService.registerUser(user);
    return ResponseEntity.ok(savedUser); // Returns savedUser containing plaintext password
}
```

Remediation: Annotate `password` in `User.java` with `@JsonProperty(access = JsonProperty.Access.WRITE_ONLY)` or return a specialized `UserResponseDTO`.

HIGH**Unprotected Endpoints & Flask Debug Mode on 0.0.0.0**

CWE-306 / CWE-489

SecurityConfig.java (Lines 12-23) | ml-service/app.py (Lines 430-436)

- **Spring Security:** `.authorizeHttpRequests(auth -> auth.anyRequest().permitAll())` makes all endpoints public. No login or JWT token generation/validation exists.
- **Flask ML Service:** `app.run(host="0.0.0.0", port=5000, debug=True)` exposes the interactive Werkzeug debugger on all interfaces, enabling remote execution risks on shared networks.

Remediation: Set `debug=False` in production/exhibition, restrict CORS, and implement a dedicated authentication controller.

4. Detailed Findings: Build, Configuration & Architecture

CRITICAL

Test Suite Fails Build Due to Package Mismatch

BUILD BLOCKER

src/test/java/com/cybershield/CybershieldAiApplicationTests.java

The test suite was copied or retained from an older prototype named "CyberShield". The test class uses `package com.cybershield;`, while the main application class is under `package com.gillnet;`. Running `./mvnw test` crashes:

```
[ERROR] Tests run: 1, Failures: 0, Errors: 1, Skipped: 0
[ERROR] CybershieldAiApplicationTests -- Time elapsed: 0.701 s <<< ERROR!
java.lang.IllegalStateException: Unable to find a @SpringBootConfiguration by searching packages upwards from the
```

Remediation: Move the test file to `src/test/java/com/gillnet/GillnetAiApplicationTests.java` and update the package declaration to `package com.gillnet;`.

HIGH

Missing `application.properties` & MongoDB Configuration

CONFIGURATION

GillNet-AI-main/src/main/resources/ (Directory Missing)

There is no configuration file anywhere in the Spring Boot project. Spring Data MongoDB falls back to `mongodb://localhost:27017/test`. Without this file:

- Connecting to MongoDB Atlas or custom databases fails.
- Server port cannot be customized.
- External ML service URL (`http://localhost:5000`) cannot be externalized or injected.

Remediation: Create `src/main/resources/application.properties` with `spring.data.mongodb.uri`, `server.port`, and ML service URL.

HIGH

Complete Absence of Core Cybersecurity Endpoints

SPECIFICATION GAP

PROJECT_CONTEXT.md Section 3 vs com.gillnet.controller.*

According to the project requirements, Spring Boot is the central integration controller for all analysis tools. However, **none** of the following required endpoints exist:

REQUIRED SPECIFICATION	EXPECTED ENDPOINT	CURRENT STATUS
Phishing URL Analyzer	<code>POST /api/url/analyze</code>	NOT IMPLEMENTED
Scam Message Analyzer	<code>POST /api/message/analyze</code>	NOT IMPLEMENTED
Password Strength Analyzer	<code>POST /api/password/analyze</code>	NOT IMPLEMENTED
AI Security Assistant	<code>POST /api/chat</code>	NOT IMPLEMENTED
Scan History API	<code>GET /api/history</code>	NOT IMPLEMENTED

Remediation: Implement the missing controllers and write a `UrlScanService` in Spring Boot using `RestClient` to call the Flask ML service on port 5000.

5. Detailed Findings: Python ML Service Logic Bugs

HIGH Path-Dependent Model Loading Crashes Service

RUNTIME CRASH

GillNet-AI-main/ml-service/app.py (Lines 14-19)

`open("phishing_url_model.pkl", "rb")` relies on the current shell working directory. If executed from the project root (e.g. `python ml-service/app.py`), the service immediately crashes with `FileNotFoundError`.

```
# Current flawed code:
with open("phishing_url_model.pkl", "rb") as f:
    model = pickle.load(f)

# Corrected robust approach:
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
with open(os.path.join(BASE_DIR, "phishing_url_model.pkl"), "rb") as f:
    model = pickle.load(f)
```

HIGH URL Validator Rejects Normal User Inputs

INPUT VALIDATION FLAW

GillNet-AI-main/ml-service/app.py (Lines 281-289)

The API validates URLs using `parsed = urlparse(url)` and tests `if not parsed.scheme` or `not parsed.netloc: return 400`. If a user pastes `paypal-secure.com` or `google.com` without typing `http://` or `https://`, `urlparse` sets `scheme=""` and `netloc=""`, causing the API to reject valid domains.

Remediation: Automatically prepend `https://` if the input lacks a scheme before parsing: `if "://" not in url: url = "https://" + url`.

MEDIUM Contradictory Prediction Output & Arbitrary Risk Score

LOGIC CONTRADICTION

GillNet-AI-main/ml-service/app.py (Lines 348-371)

1. If the ML model classifies a URL as `PHISHING`, but none of the 9 specific heuristic checks triggered, the service falls back to: `"reasons": ["No major suspicious URL characteristics detected"]`. This produces an API response telling the user the URL is Phishing while simultaneously stating no suspicious traits were found!
2. The risk score is hardcoded to static values (80, 55, or 15) and completely ignores `confidence` from `predict_proba`.

```
// Actual contradictory API response example:
{
  "prediction": "PHISHING",
  "riskScore": 80,
  "reasons": ["No major suspicious URL characteristics detected"],
  "recommendation": "Do not open this website or enter personal information."
}
```

GillNet-AI-main/ml-service/app.py (Lines 94–143)

- **Subdomain Check (L134–142):** Counts dots in hostname. Legitimate domains with ccTLDs (e.g. `www.bbc.co.uk`) have 3 dots and are falsely penalized as phishing.
- **At Symbol Check (L97–101):** Checks "@" in url across query strings. Legitimate login URLs containing emails (e.g. `?email=user@domain.com`) are penalized.
- **Double Slash Check (L108–118):** Flags URLs where redirect parameters contain `https://`.
- **Dead Code (L29–34):** `SUSPICIOUS_KEYWORDS` is declared with 14 terms but never referenced anywhere.

6. Prioritized Remediation Roadmap

To prepare GillNet AI for a successful exhibition demo, follow this phased action plan:

Phase 1: Urgent Security & Build Fixes (Day 1)

- [] **Fix Test Suite:** Rename test package to `com.gillnet` in `src/test/java/com/gillnet/GillnetAiApplicationTests.java` to get green builds.
- [] **Password Hashing:** Define `BCryptPasswordEncoder` in `SecurityConfig.java` and hash passwords in `UserService.registerUser()`.
- [] **Prevent Credential Leaks:** Create a `UserResponseDTO` or annotate `password` with `@JsonProperty(access = WRITE_ONLY)`. Remove or restrict `/api/users/email/{email}`.
- [] **Add Configuration:** Create `src/main/resources/application.properties` with MongoDB credentials and server port.

Phase 2: Core Cybersecurity REST Endpoints (Days 2–4)

- [] **Implement URL Controller:** Create `/api/url/analyze` in Spring Boot that delegates requests to `http://localhost:5000/api/v1/url/check` via `RestClient`.
- [] **Implement Message Analyzer:** Create `/api/message/analyze` implementing heuristic scam rules on Spring Boot.
- [] **Implement Password Analyzer:** Create `/api/password/analyze` checking entropy, character sets, and common passwords (ensuring submitted passwords are never logged or stored).
- [] **History Storage:** Create a `ScanRecord` MongoDB document and `ScanHistoryRepository` to persist user scans.

Phase 3: Python ML Service Hardening (Day 5)

- [] **Fix Path Loading:** Use `os.path.dirname(__file__)` to resolve model file paths.
- [] **Prepend Schemes:** Auto-prepend `https://` to user inputs before `urlparse`.
- [] **Dynamic Risk Score:** Calculate `riskScore` dynamically using `confidence * 100`.
- [] **Resolve Reason Contradiction:** Ensure fallback reason for phishing predictions states: *"Model identified structural phishing features."*

Phase 4: Frontend-Backend Integration (Days 6–7)

- [] Replace mock heuristics in `Dashboard.tsx` with active `fetch()` calls to the Spring Boot REST API.
- [] Add error banners and loading spinners for live API analysis.